STUDENT NAME: **YASH CHAUHAN**
STUDENT ROLL NUMBER: **28**
SECTION: CSE **C-1**
STUDENT REG NUMBER: **230905194**
WEEK NO: 9
DATE: **17TH MARCH 2026**
LAB8 – RECURSIVE DESCENT PARSER

Q1. Grammar 7.1 with given production.

Ans -

Token.h

```
#ifndef TOKEN_H
#define TOKEN_H

#include <stdio.h>
#include <ctype.h>
#include <string.h>

struct token {
char token_name[50];
int index;
unsigned int row, col;
char type[30];
};

extern char *keywords[];
extern int keywordCount;

int isKeyword(char *);
int isArithmetic(char *);
int isRelational(char *);
int isLogical(char *);
int isSpecialSymbol(char);
```

```
#endif
```

Lexer.c

```
#include "token.h"
#include <stdlib.h>

FILE *fp;
unsigned int row = 1, col = 0;

char *keywords[] = {
"int","float","char","double","void",
"if","else","while","for","return"
};
int keywordCount = 10;

int checkKeyword(char *lex, int i) {
if (i == keywordCount) return 0;
if (strcmp(lex, keywords[i]) == 0) return 1;
return checkKeyword(lex, i + 1);
}

int isKeyword(char *lex) {
return checkKeyword(lex, 0);
}

int isArithmetic(char *lex) {
return (!strcmp(lex,"+") || !strcmp(lex,"-") ||
!strcmp(lex,"*") || !strcmp(lex,"/") ||
!strcmp(lex,"%"));
}

int isRelational(char *lex) {
return (!strcmp(lex,"<") || !strcmp(lex,">") ||
!strcmp(lex,"<=") || !strcmp(lex,">=") ||
!strcmp(lex,"==") || !strcmp(lex,"!="));
}

int isLogical(char *lex) {
```

```
return (!strcmp(lex,"&&") || !strcmp(lex,"||") ||
!strcmp(lex,"!"));
}

int isSpecialSymbol(char ch) {
return (ch=='('||ch==')'||ch=='{'||ch=='}'||
ch==';'||ch==','||ch=='['||ch==']');
}

void buildIdentifier(char *buf, int *i) {
char c = fgetc(fp);
if (isalnum(c) || c == '_') {
buf[(*i)++] = c;
col++;
buildIdentifier(buf, i);
} else {
ungetc(c, fp);
}
}

void buildNumber(char *buf, int *i) {
char c = fgetc(fp);
if (isdigit(c)) {
buf[(*i)++] = c;
col++;
buildNumber(buf, i);
} else {
ungetc(c, fp);
}
}

int getNextToken(struct token *t) {
char c = fgetc(fp);
if (c == EOF) return 0;

col++;

if (c == '\n') {
row++; col = 0;
return getNextToken(t);
}

if (isspace(c))
```

```
return getNextToken(t);

int i = 0;

if (isalpha(c) || c == '_') {
t->row = row; t->col = col;
t->token_name[i++] = c;
buildIdentifier(t->token_name, &i);
t->token_name[i] = '\0';

if (isKeyword(t->token_name))
strcpy(t->type, "KEYWORD");
else
strcpy(t->type, "id");

return 1;
}

if (isdigit(c)) {
t->row = row; t->col = col;
t->token_name[i++] = c;
buildNumber(t->token_name, &i);
t->token_name[i] = '\0';
strcpy(t->type, "num");
return 1;
}

t->row = row; t->col = col;
t->token_name[i++] = c;

char next = fgetc(fp);
if (next == '=' || (c=='&' && next=='&') || (c=='|' && next=='|')) {
t->token_name[i++] = next;
} else {
ungetc(next, fp);
}

t->token_name[i] = '\0';

if (isArithmetic(t->token_name))
strcpy(t->type, "ARITHMETIC OPERATOR");
else if (isRelational(t->token_name))
```

```
strcpy(t->type, "RELATIONAL OPERATOR");
else if (!strcmp(t->token_name, "="))
strcpy(t->type, "ASSIGNMENT OPERATOR");
else if (isLogical(t->token_name))
strcpy(t->type, "LOGICAL OPERATOR");
else if (isSpecialSymbol(c))
strcpy(t->type, "SPECIAL SYMBOL");
else
strcpy(t->type, "UNKNOWN");

return 1;
}
```

Q1.c

```
#include "token.h"
#include <stdlib.h>
#include <stdio.h>

extern FILE *fp;

struct token lookahead;
int error_count = 0;

int getNextToken(struct token *);

void printToken(struct token t) {
printf("<%s,%s,%u,%u>\n", t.token_name, t.type, t.row, t.col);
}

void skipTokens() {
if (!strcmp(lookahead.token_name, ";") ||
!strcmp(lookahead.token_name, "}"))
return;

if (!getNextToken(&lookahead))
return;

skipTokens();
}
```

```
void error_expected(const char *exp) {
printf("Error at row %u, col %u: expected %s, found %s\n",
lookahead.row, lookahead.col, exp, lookahead.token_name);

error_count++;
skipTokens();

if (!strcmp(lookahead.token_name, ";"))
getNextToken(&lookahead);
}

void match(const char *t) {
if (!strcmp(t,"id") || !strcmp(t,"num")) {
if (!strcmp(lookahead.type,t)) {
printToken(lookahead);
getNextToken(&lookahead);
} else error_expected(t);
} else {
if (!strcmp(lookahead.token_name,t)) {
printToken(lookahead);
getNextToken(&lookahead);
} else error_expected(t);
}
}

void expn(); void eprime();
void simple_expn(); void seprime();
void term(); void tprime();
void factor();

void expn() {
simple_expn();
eprime();
}

void eprime() {
if (!strcmp(lookahead.token_name,"==") ||
!strcmp(lookahead.token_name,"!=") ||
!strcmp(lookahead.token_name,"<") ||
!strcmp(lookahead.token_name,">") ||
!strcmp(lookahead.token_name,"<=") ||
!strcmp(lookahead.token_name,">=")) {
```

```
match(lookahead.token_name);
simple_expn();
}
}

void simple_expn() {
term();
seprime();
}

void seprime() {
if (!strcmp(lookahead.token_name,"+") ||
!strcmp(lookahead.token_name,"-")) {
match(lookahead.token_name);
term();
seprime();
}
}

void term() {
factor();
tprime();
}

void tprime() {
if (!strcmp(lookahead.token_name,"*") ||
!strcmp(lookahead.token_name,"/") ||
!strcmp(lookahead.token_name,"%")) {
match(lookahead.token_name);
factor();
tprime();
}
}

void factor() {
if (!strcmp(lookahead.type,"id"))
match("id");
else if (!strcmp(lookahead.type,"num"))
match("num");
else
error_expected("id or num");
}
```

```
void identifier_list() {
match("id");

if (!strcmp(lookahead.token_name,"[")) {
match("[");
match("num");
match("]");
}

if (!strcmp(lookahead.token_name,",")) {
match(",");
identifier_list();
}
}

void declarations() {
if (!strcmp(lookahead.token_name,"int") ||
!strcmp(lookahead.token_name,"char")) {
match(lookahead.token_name);
identifier_list();
match(";");
declarations();
}
}

void assign_stat() {
match("id");

if (!strcmp(lookahead.token_name,"[")) {
match("[");
match("num");
match("]");
}

match("=");
expn();
match(";");
}

void statement() {
assign_stat();
}
```

```
void statement_list() {
if (!strcmp(lookahead.type,"id")) {
statement();
statement_list();
}
}

void program() {
match("main");
match("(");
match(")");
match("{");

declarations();
statement_list();

match("}");
}

int main() {
fp = stdin;

if (!getNextToken(&lookahead)) {
printf("Empty input\n");
return 0;
}

program();

if (error_count == 0)
printf("Parsing successful\n");
else
printf("Parsing completed with %d error(s)\n", error_count);

return 0;
}
```

OUTPUT -

Test case 1 with multiple error -

```
Lab8 > input.txt
1   main() {
2       int a b[10]
3       a = ;
4       b[2 = a + * 4
5   }
```

```
6CSEC1@dbl-29:~/Desktop/Yash Chauhan$ cd Lab8
6CSEC1@dbl-29:~/Desktop/Yash Chauhan/Lab8$ gcc lexer.c q1.c -o q1
6CSEC1@dbl-29:~/Desktop/Yash Chauhan/Lab8$ ./q1 < input.txt
<main,id,1,1>
<(,SPECIAL SYMBOL,1,5>
<),SPECIAL SYMBOL,1,6>
<{,SPECIAL SYMBOL,1,8>
<int,KEYWORD,2,5>
<a,id,2,9>
Error at row 2, col 11: expected ;, found b
<b,id,4,5>
<[,SPECIAL SYMBOL,4,6>
<2,num,4,7>
Error at row 4, col 9: expected ], found =
Error at row 5, col 1: expected =, found }
Error at row 5, col 1: expected id or num, found }
Error at row 5, col 1: expected ;, found }
<},SPECIAL SYMBOL,5,1>
Parsing completed with 5 error(s)
6CSEC1@dbl-29:~/Desktop/Yash Chauhan/Lab8$
```

Testcase 2 with no errors

```
Lab8 > input.txt
1   main() {
2       int a, b[10];
3       a = 5;
4       b[2] = a + 3 * 4;
5   }
```

```
6CSEC1@dbl-29:~/Desktop/Yash Chauhan/Lab8$ gcc lexer.c q1.c -o q1
6CSEC1@dbl-29:~/Desktop/Yash Chauhan/Lab8$ ./q1 < input.txt
<main,id,1,1>
<(,SPECIAL SYMBOL,1,5>
<),SPECIAL SYMBOL,1,6>
<{,SPECIAL SYMBOL,1,8>
<int,KEYWORD,2,5>
<a,id,2,9>
<,,SPECIAL SYMBOL,2,10>
<b,id,2,12>
<[,SPECIAL SYMBOL,2,13>
<10,num,2,14>
<],SPECIAL SYMBOL,2,16>
<;,SPECIAL SYMBOL,2,17>
<a,id,3,5>
<=,ASSIGNMENT OPERATOR,3,7>
<5,num,3,9>
<;,SPECIAL SYMBOL,3,10>
<b,id,4,5>
<[,SPECIAL SYMBOL,4,6>
<2,num,4,7>
<],SPECIAL SYMBOL,4,8>
<=,ASSIGNMENT OPERATOR,4,10>
<a,id,4,12>
<+,ARITHMETIC OPERATOR,4,14>
<3,num,4,16>
<*,ARITHMETIC OPERATOR,4,18>
<4,num,4,20>
<;,SPECIAL SYMBOL,4,21>
<},SPECIAL SYMBOL,5,1>
Parsing successful
6CSEC1@dbl-29:~/Desktop/Yash Chauhan/Lab8$
```